OK, so what can you do about this nasty race condition? The answer is to use an atomic compare/exchange operation at step 3 to ensure that `head` hasn't been modified since you read it in step 2. If it has, you can loop and try again. The following listing shows how you can implement a thread-safe `push()` without locks.

**Listing 7.2. Implementing `push()` without locks**

```
template<typename T>
class lock_free_stack
{
private:
    struct node
    {
        T data;
        node* next;

        node(T const& data_):    <-(1)
            data(data_)
        {}
    };

    std::atomic<node*> head;
public:
    void push(T const& data)
    {
        node* const new_node=new node(data);    <-(2)
        new_node->next=head.load();    <-(3)
        while(!head.compare_exchange_weak(new_node->next,new_node));    <-(4)
    }
};
```

This code neatly matches the three-point plan from above: create a new node ❷, set the node's `next` pointer to the current `head` ❸, and set the `head` pointer to the new node ❹. By populating the data in the `node` structure itself from the `node` constructor ❶, you've ensured that the node is ready to roll as soon as it's constructed, so that's the easy problem down. Then you use `compare_exchange_weak()` to ensure that the `head` pointer still has the same value as you stored in `new_node->next` ❸, and you set it to `new_node` if so. This bit of code also uses a nifty part of the compare/exchange functionality: if it returns `false` to indicate that the comparison failed (for example, because `head` was modified by another thread), the value supplied as the first parameter (`new_node->next`) is updated to the current value of `head`. You therefore don't have to reload